4/7

# ^사물컴퍼니

- 자주쓰는 AI소개
- git/github
- VS code with codex
- 팀빌딩 안내



| AI | 사용 |
| --- | --- |
| GPT pro , claude opus | 보고서, 글쓰기, 추론 |
| Claude code, cursor, codex | 코딩 |
| Gemini 나노바나나 -> 3 Flash | 이미지 |
| Gemini pro / plus | 방대한 자료, 구글링, 유튜브 |
| Perplexity | 논문, 사실적 자료 (출처 표기) |
| gamma | Ppt 생성 |




2005년 Linux의 아버지인 리누스 토발즈가 리눅스 커널 프로젝트를 위해 개발한 분산식 버전 관리 시스템이다.

# Git

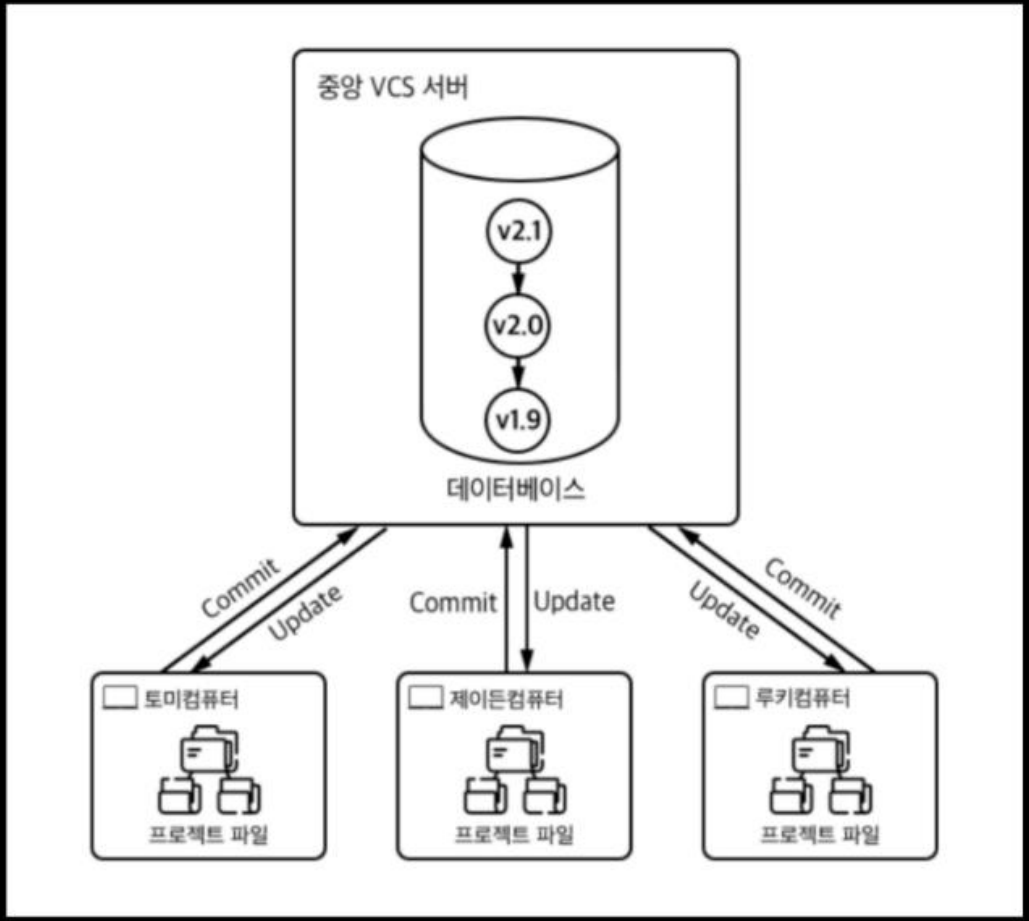
- 버전 관리 시스템 VCS: Version Control System




파일을 시간별로 기록했다가 필요 시 특정 시점의 버전을 다시 꺼내올 수 있는 시스템

# 버전 관리 시스템

## 중앙 집중식 버전 관리




장점

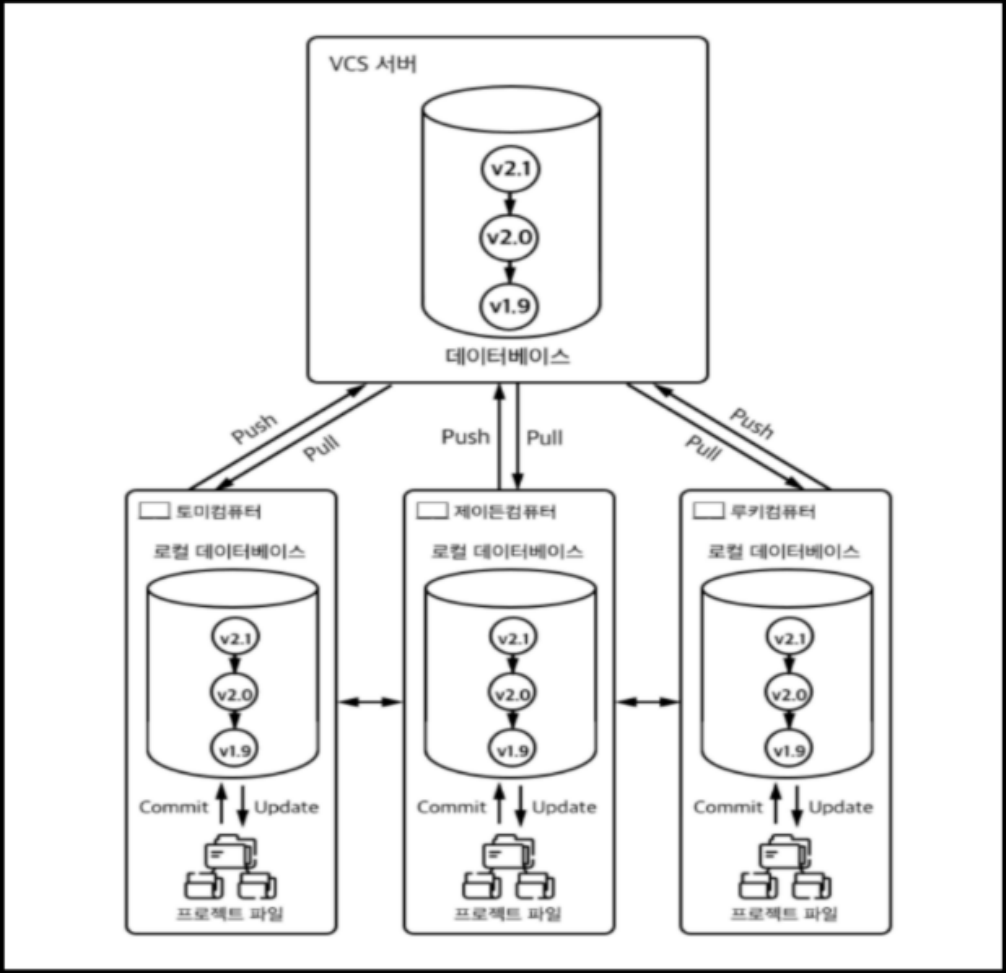
- 모두 누가 무엇을 하고 있는지 알 수 있다.
- 관리자는 누가 무엇을 할지 꼼꼼하게 관리할 수 있다.
- 모든 클라이언트의 로컬 데이터베이스를 관리하는 것보다 VCS 하나를 관리하기가 훨씬 쉽다.

단점

- 만약 서버가 한 시간 동안 다운되면 그동안 아무도 다른 사람과 협업할 수 없고 사람들이 하는 일을 백업할 방법도 없다.
- 중앙 데이터베이스가 있는 하드디스크에 문제가 생기면 프로젝트의 모든 히스토리를 잃는다.

# 버전 관리 시스템

## 분산 버전 관리 시스템

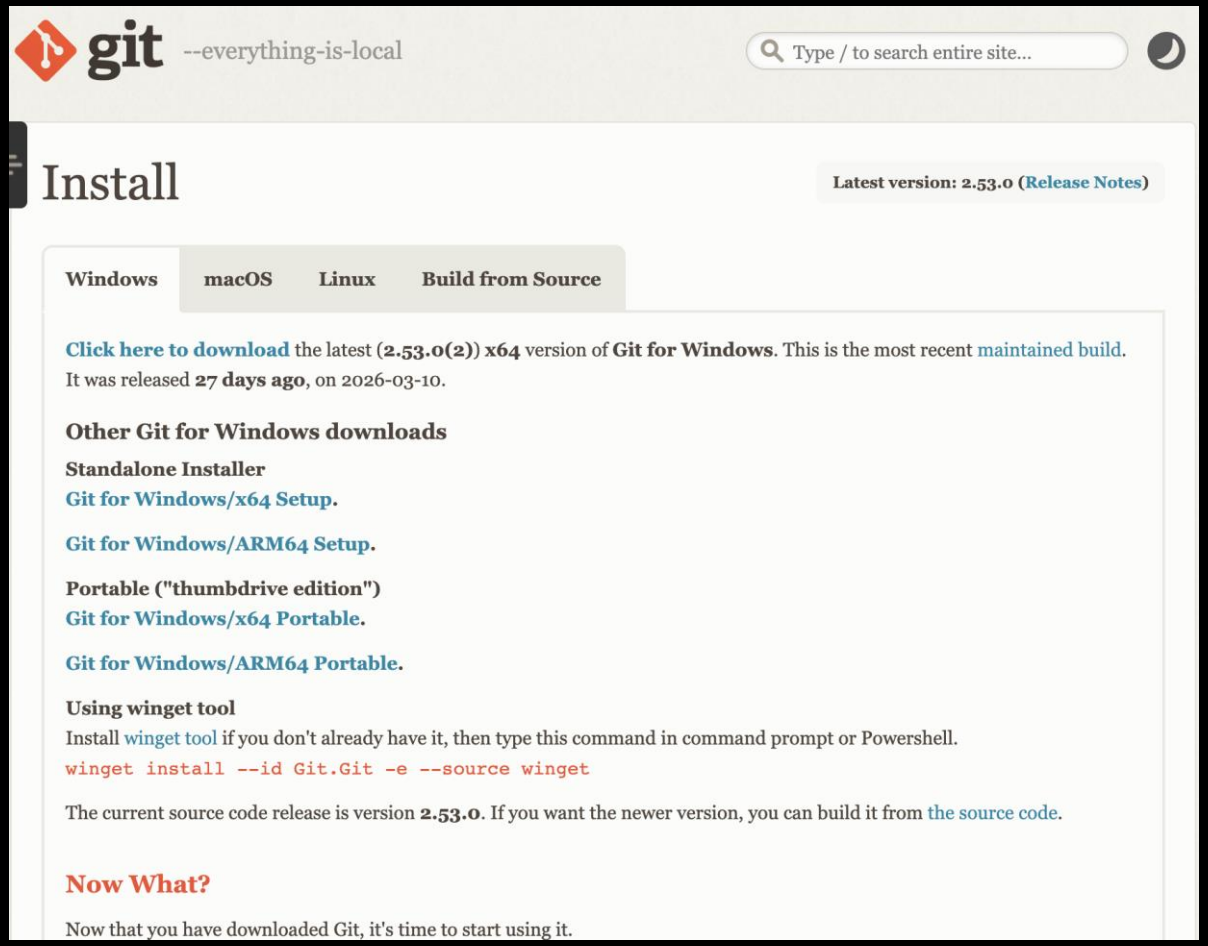



- 클라이언트 중에서 아무거나 골라도 서버를 복원할 수 있다.
- Clone은 모든 데이터를 가진 진정한 백업이다.
- 평소 작업은 로컬 저장소에서 수행, 파일을 변경하고 이력을 기록함
- 다른 사람과 동기화가 필요할 때만
  원격 서버에 접속해서 변경 이력을 원격저장소에 반영함

# Git

## Git 설치

https://git-scm.com/downloads



# Git -> vs code에 연동하기

# VS code + git

## 윈도우:

WSL -> window subsystem for Linux

## MacOS:

유닉스 기반

## Git초기설정 (VS code terminal)

- 깃 버전 확인

git --version

- 이름과 이메일 등록 (한 번만)

git config --global user.name "본인이름"

git config --global user.email "이메일주소"

- 설정 확인

git config --list

# VS code + git

## 윈도우:

(Powershell)

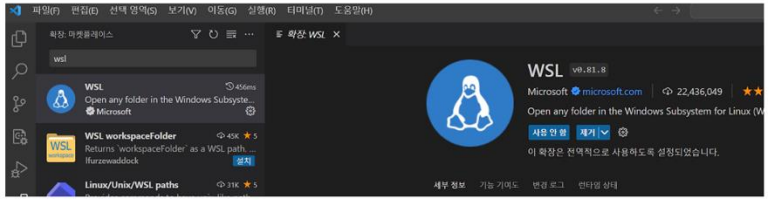
1. wsl --install
2. wsl
   (실행후 계정등록-
   비번 칠때 안보임)

(WSL)

2. sudo apt update
3. sudo apt install git
4. git --version

### VSCode에서 WLS2 연동

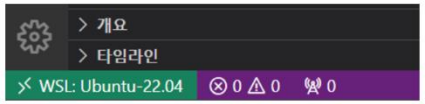
- vscode 실행 후 "확장" 탭으로 이동하여 "WSL"을 설치합니다.



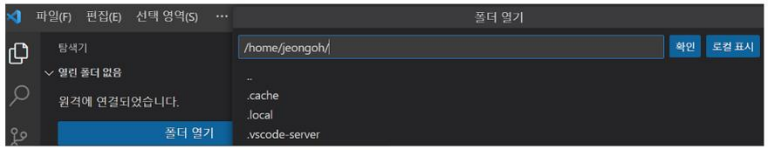

- vscode에서 "ctrl + shift + p" 명령 입력 후 "WSL: WSL에 연결"을 선택합니다.




- WSL 연결에 완료되면 vscode 좌측 하단에 다음과 같이 표시됩니다.




- "폴더 열기"를 선택하여 작업할 폴더를 선택합니다.




- 이후 필요한 패키지를 설치하고 개발 환경을 구성하여 작업하면 됩니다.

## MacOS:

(terminal)

1. xcode-select –install (개발 도구 설치)
2. /bin/bash -c
"$(curl -fsSL https://raw.githubusercontent.com
Homebrew/install/HEAD/install.sh)"
3. brew install git
4. git --version

# Github

# Github



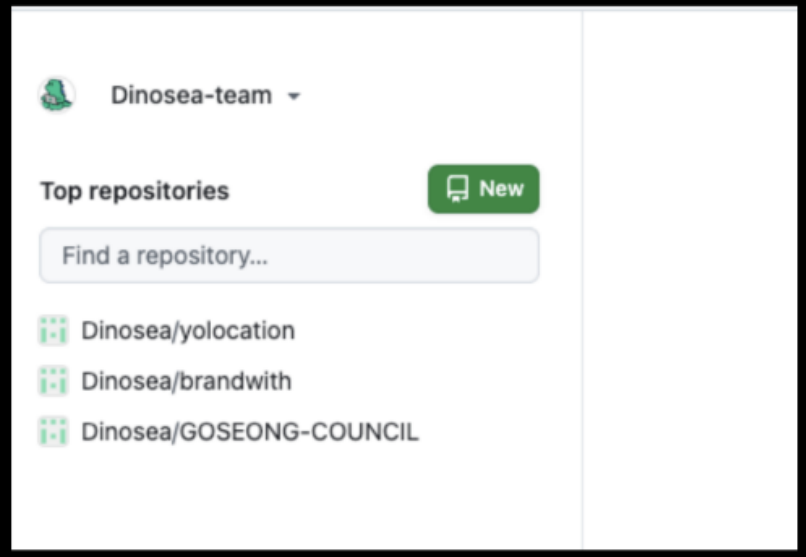

- Git의 저장소 = Github

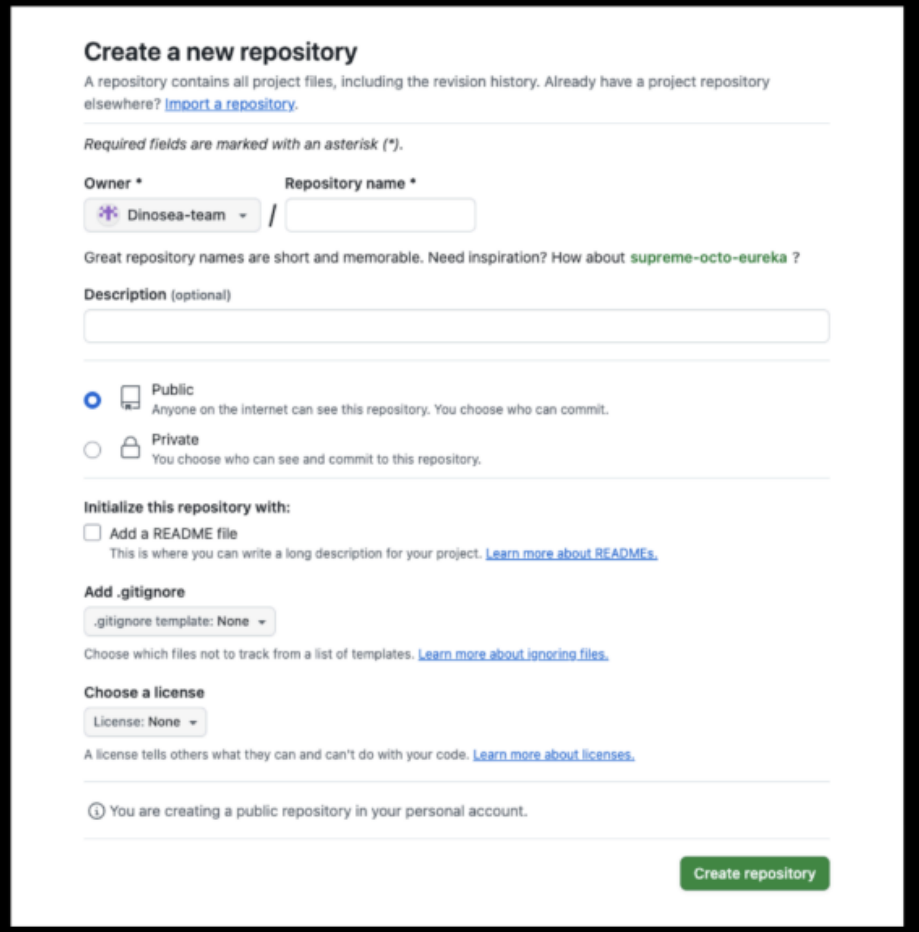
- git 명령어로 저장한 파일들이 허브에 저장

- 깃헙의 저장소 = Repository 레포지토리

# Github

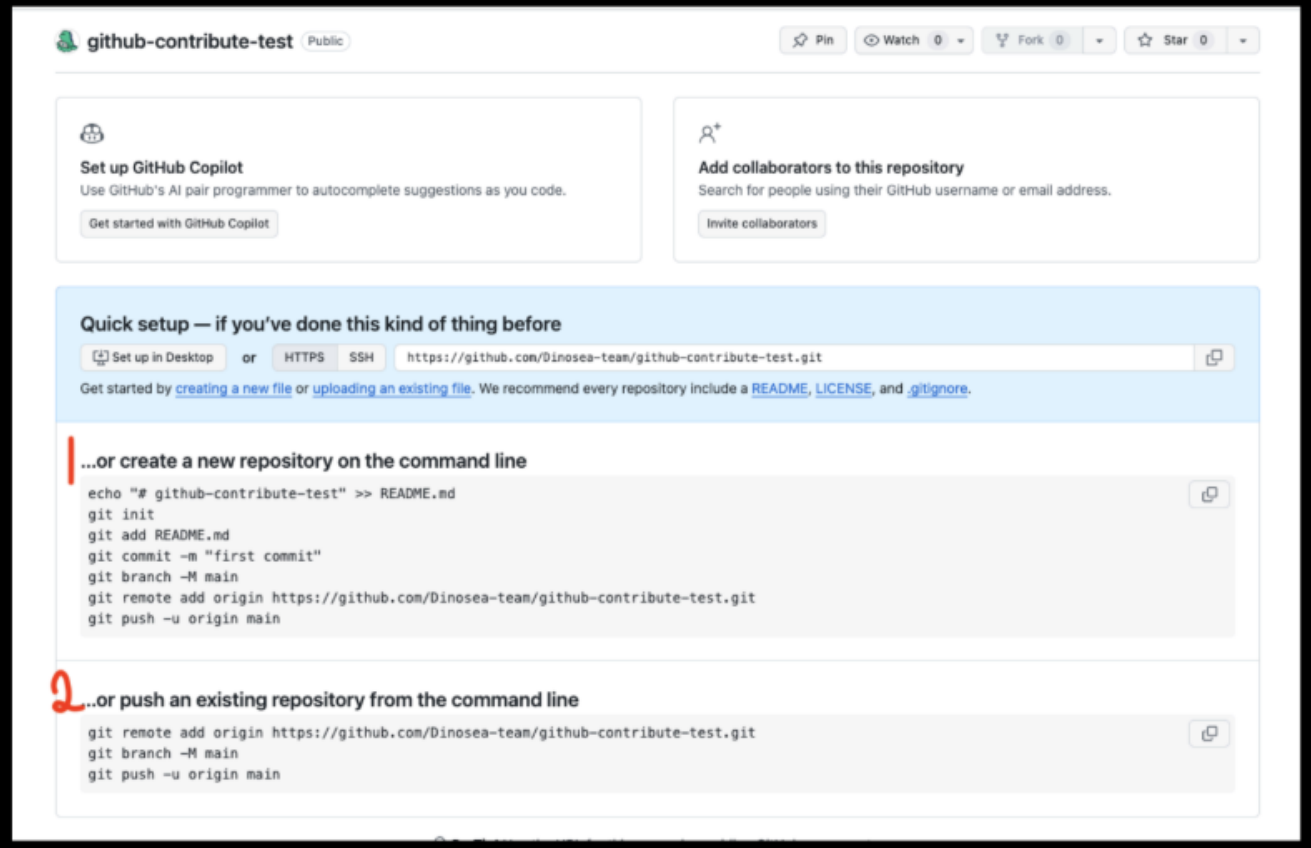
## 원격 저장소 만들기



# Github

## 원격 저장소 만들기




1. 새로운 공간에 위 저장소를 연결시킬 때
2. 기존에 생성한 공간에 위 저장소를 연결할 때

git remote add origin <url>

git push -u origin main

# Github

## 저장소 동기화하며 작업하기

개인 노트북으로 작업하는 곳 = 오피스에서 작업하는 곳

Git push & Git pull

# SSH

- SSH는 비밀번호 대신 열쇠로 인증하는 방식
- **GitHub SSH**: GitHub에 인증할 때 쓰는 SSH

# SSH 키 생성

1. **ssh-keygen –t ed25519 –C “you@example.com”**
2. **쭉 Enter**
3. **공개키 복사**

   **cat ~/.ssh/id_ed25519.pub**
4. **깃헙에 등록**

   **Settings -> SSH and GPG keys -> new ssh key**
5. **등록확인  ssh -T git@github.com -> yes -> Hi**

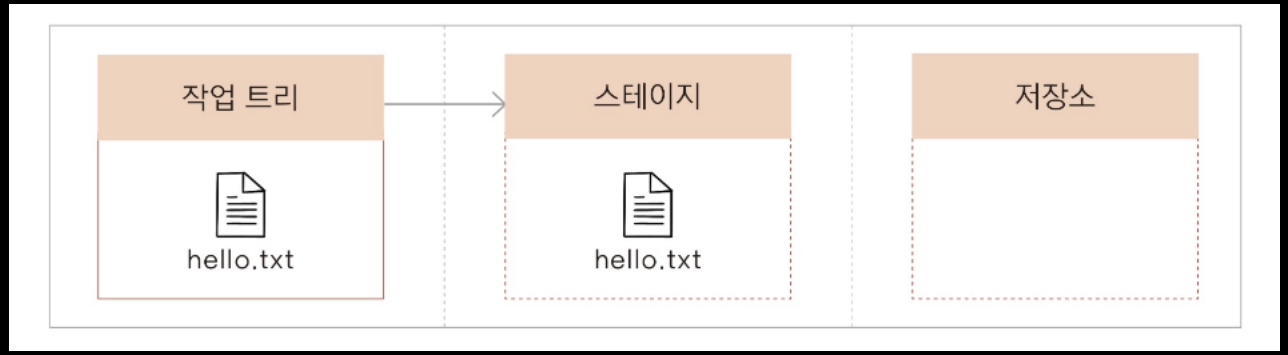
# Git 실습

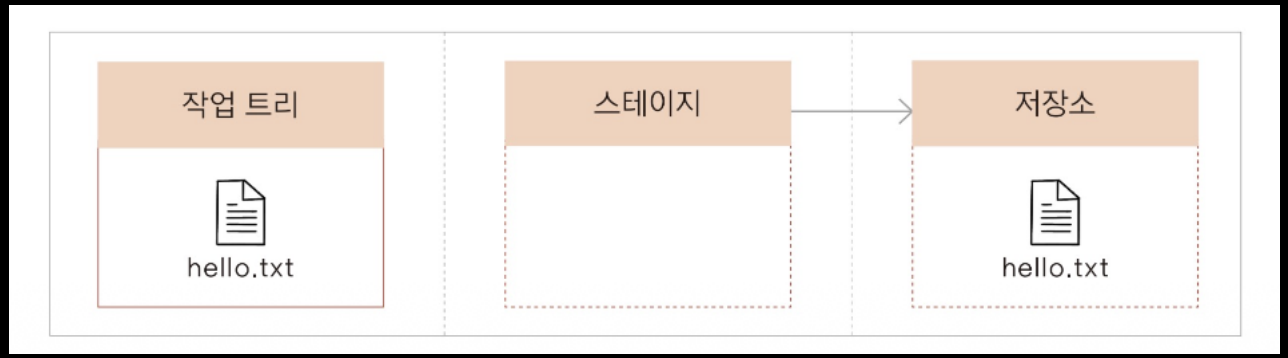
# Github 레포지토리 복제

- git clone git@github.com:USERNAME/REPO.git
- cd 레포지토리이름  (뒤로 이동은 cd ..) (자동완성은 tab치면 됨)

# Git 기본 실습

- nano hello.txt -> 아무거나 작성 후 저장
- git status
- git add  hello.txt (git add .)
- git commit -m "첫번째 깃 실습"
- git push origin main



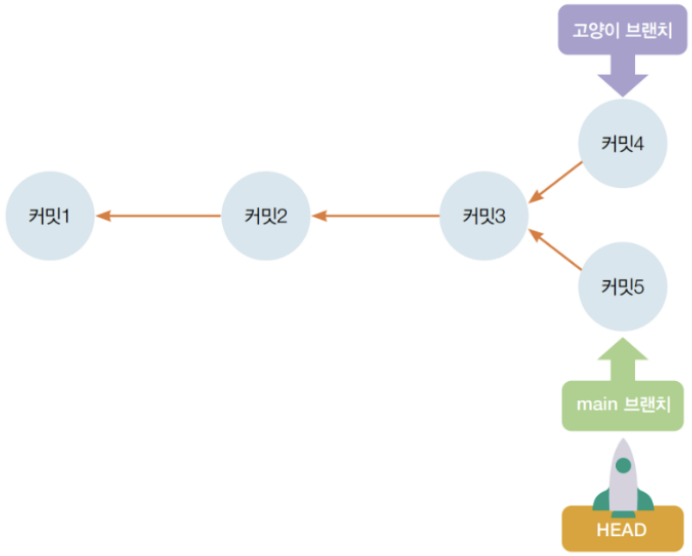




파일 생성 → add → commit → push

# branch

■ 명령어

- branch 생성 : `git branch [브랜치 이름]`
- branch 이동 : `git checkout [브랜치 이름]`
- branch 생성과 이동 : `git checkout -b [브랜치 이름]`
- branch 삭제 : `git branch -d [브랜치 이름]`
- branch 조회 : `git branch`



# Branch + merge 충돌 실습

# Branch + merge

1. 브랜치 생성 및 이동 ex) git branch cute

   git branch [브랜치 이름]

   git checkout [브랜치 이름]
2. 파일 생성 (충돌 케이스 실습을 위해 main에 동일 파일 수정)

   nano test.txt -> 수정
3. Add 후 커밋

   git add test.txt

   git commit –m "cute 브랜치에서 수정함"
4. Main 으로 이동

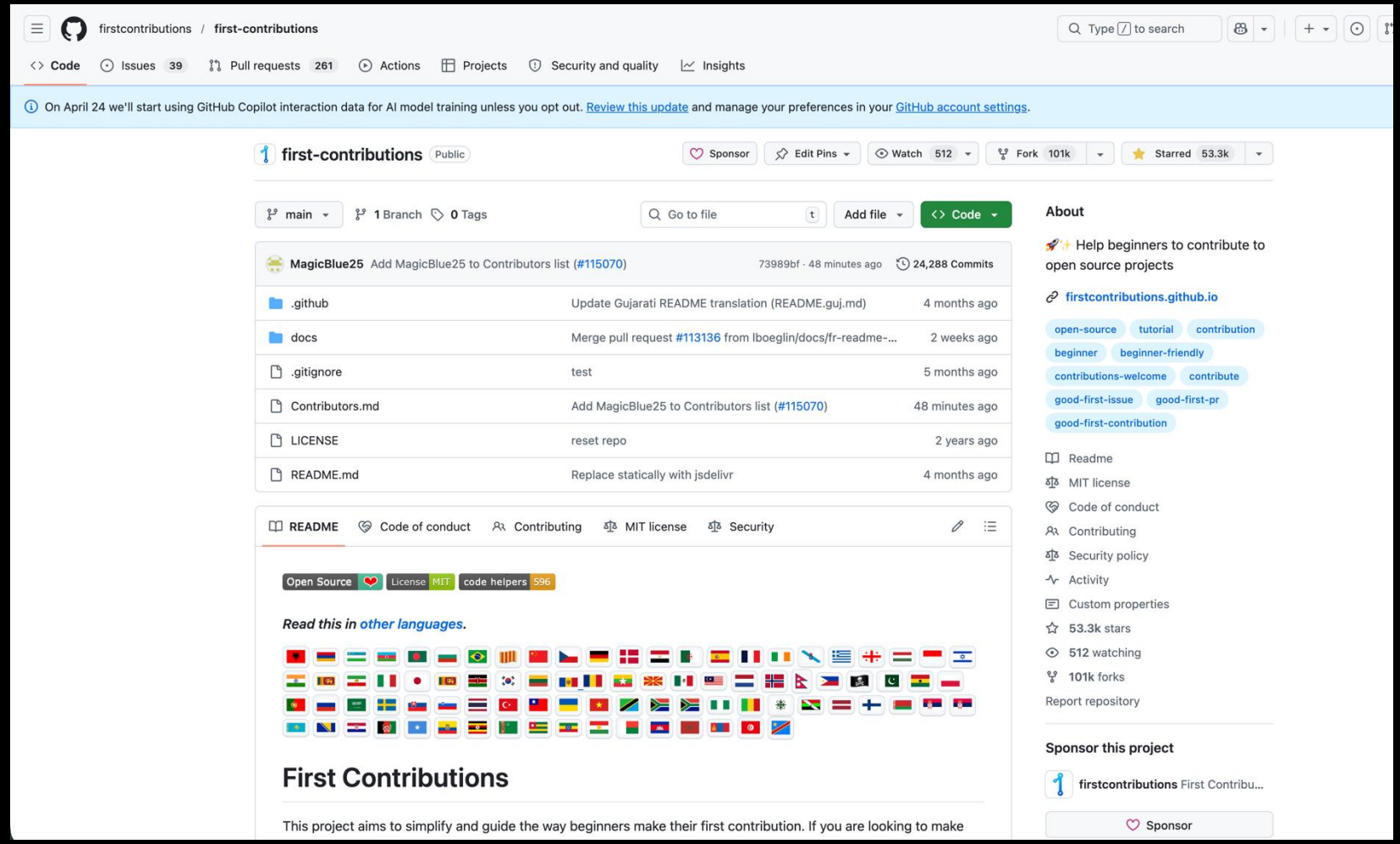
   git switch main
5. 충돌을 만들기 위해 동일 파일 수정

   nano test.txt -> 수정 후 add & commit (3번과 동일)
6. Main에서 cute 브랜치와 병합

   git merge cute
7. 충돌발생
8. 수정 후 add & commit
9. git push (origin main)

# 과제 4/28까지

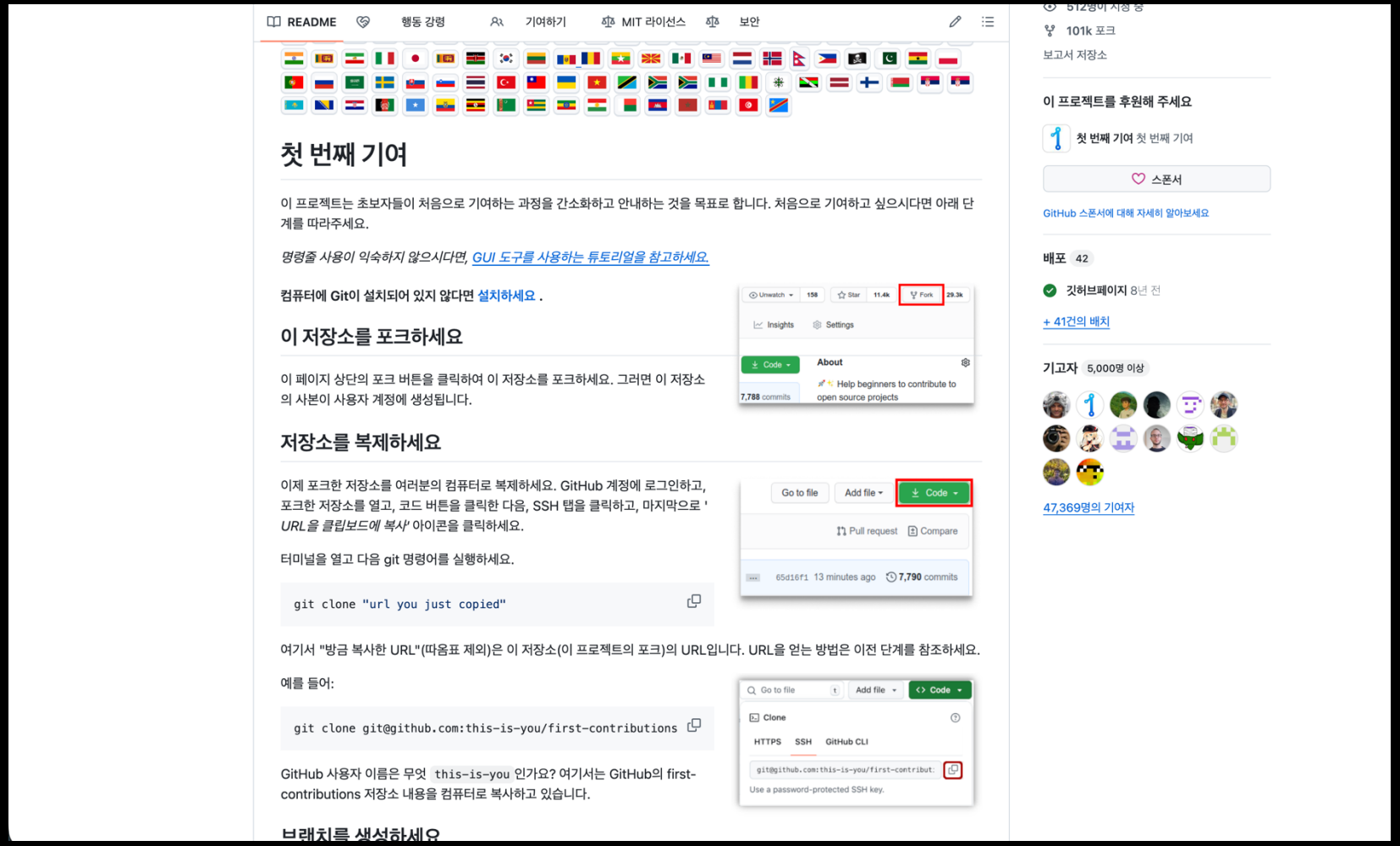
https://github.com/firstcontributions/first-contributions
접속 후 README 읽어보며 기여하기 실습 해보기



# 과제 4/28까지

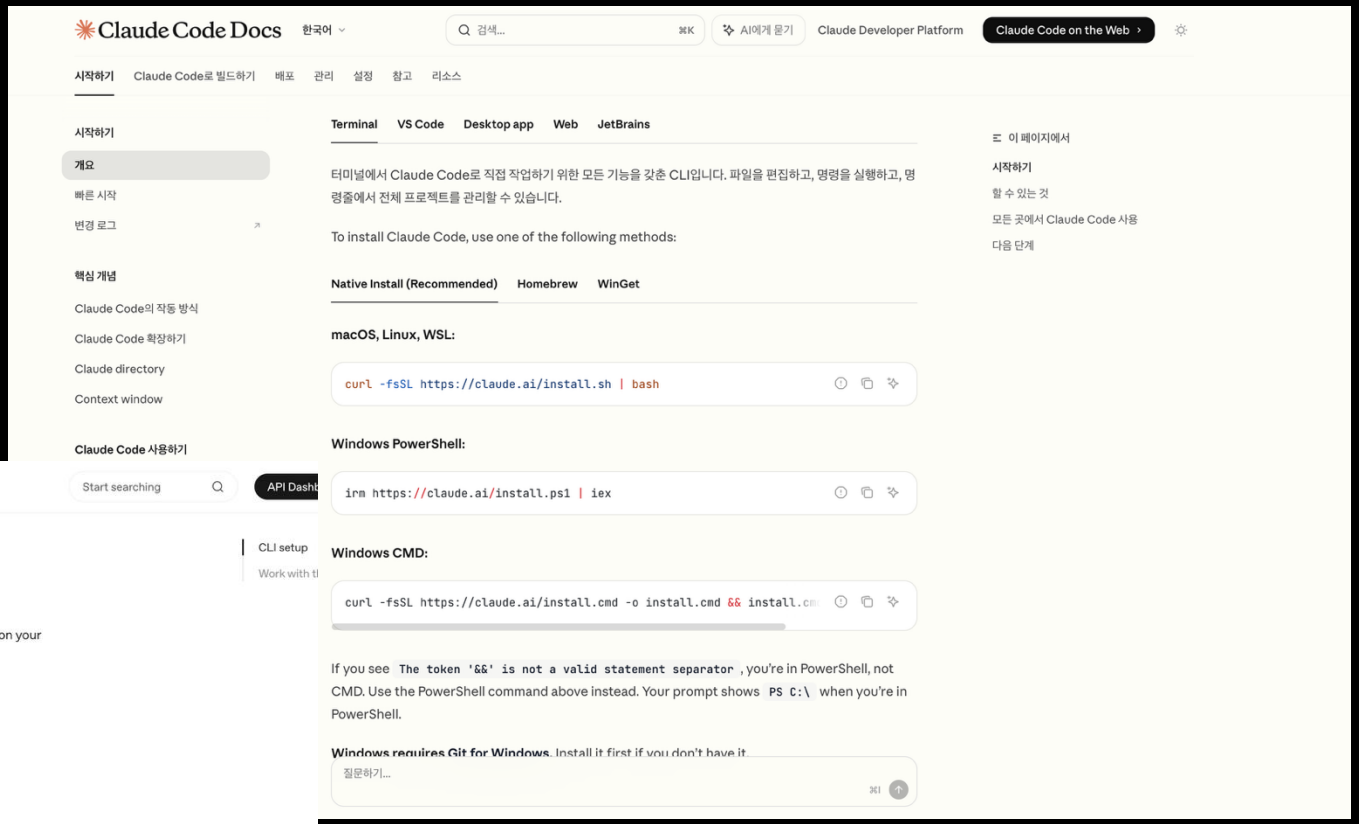
https://github.com/firstcontributions/first-contributions
접속 후 README 읽어보며 기여하기 실습 해보기

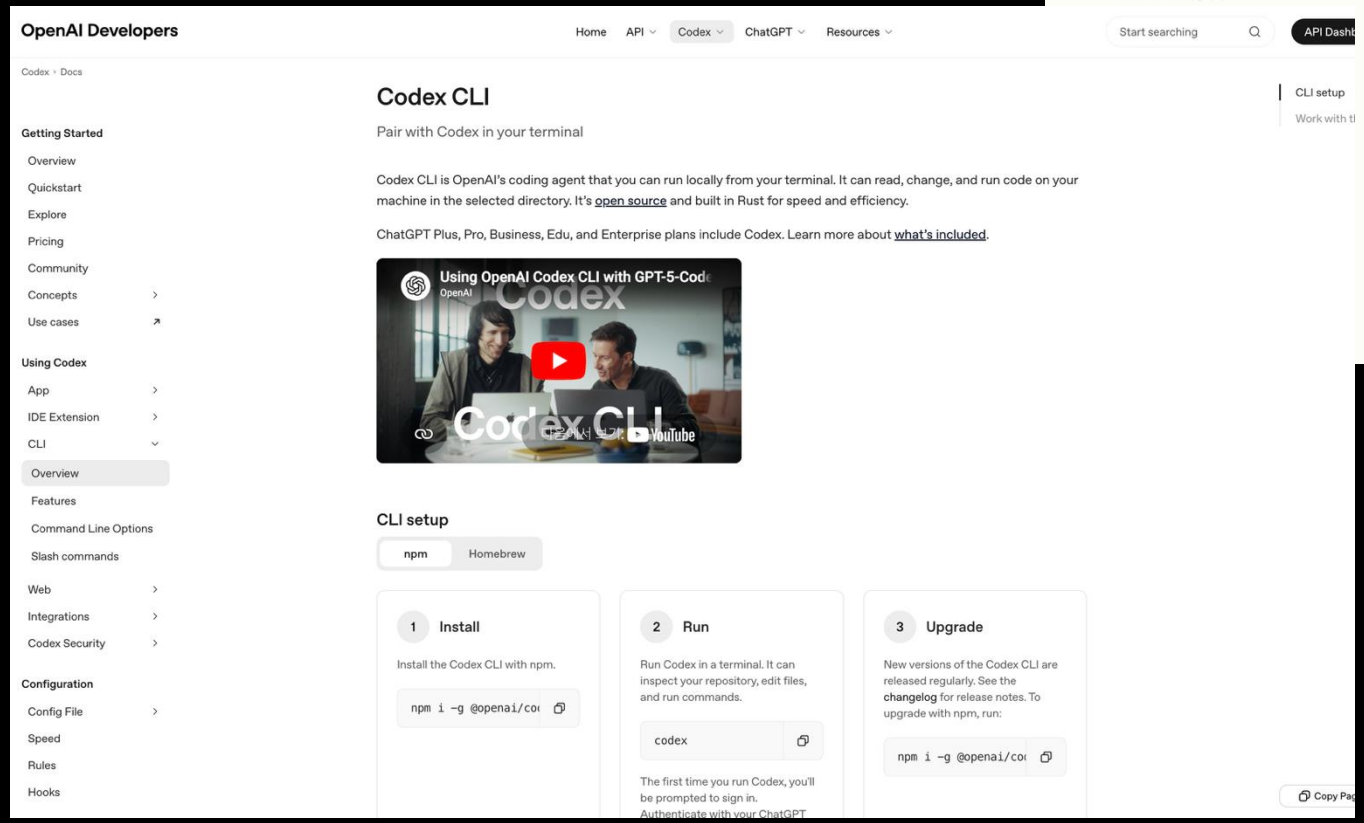
하는 방법 잘 나와있음! ->



# Terminal + AI

각자 구독하고 있는 AI 사용!
없으면 무난한 커서로 입문 or
무료버전도 가능



# Ex) codex

https://developers.openai.com/codex/cli

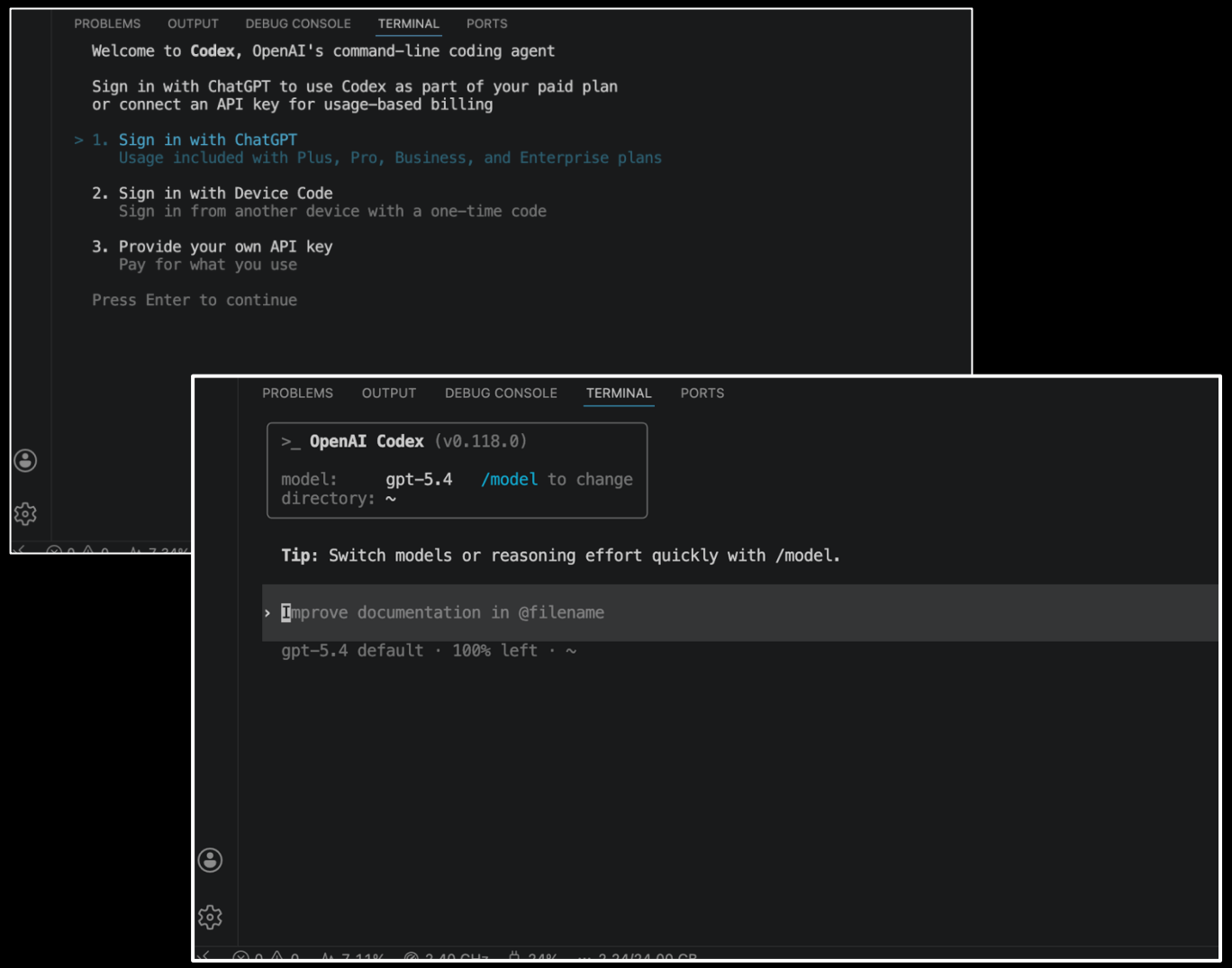
윈도우 (WSL환경에 npm없을 경우):
sudo apt update
sudo apt install nodejs npm -y

node -v
npm -v

npm i -g @openai/codex
codex

MacOS:
brew install node

설치 후 codex 입력 -> 로그인 하면 끝



<다음시간>

- 팀빌딩
- 관심 분야 구글 폼 작성
- 4/28 팀끼리 자유주제 선정 후 해산
- 5/6 프로젝트 중간 발표!!